

Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Санкт-Петербургский государственный университет промышленных технологий и дизайна»

Институт информационных технологий и автоматизации

Кафедра: Цифровых и аддитивных технологий
Направление подготовки: 09.03.03 Прикладная информатика
Профиль подготовки: Прикладная информатика в дизайне

КУРСОВАЯ РАБОТА

на тему Разработка информационно-справочной системы

по дисциплине Алгоритмизация и программирование

Исполнитель - студент
учебной группы:

2-МД-15

(группа)

Альникина Елизавета Дмитриевна

(Ф.И.О./подпись)

Руководитель курсовой
работы:

Вальштейн К.В.

(должность, ученая степень / звание, Ф.И.О./подпись)

Санкт-Петербург
2023

Министерство образования и науки Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«Санкт-Петербургский государственный университет
промышленных технологий и дизайна»

Утверждаю
Заведующий кафедрой
_____ А. В. Сошников

Институт информационных
технологий и автоматизации
Кафедра Цифровых и аддитивных
технологий

«_____» _____ 202_ г.

ЗАДАНИЕ
на курсовую работу

Студенту _____ Альтикиной Елизавете Дмитриевне

1. Тема курсовой работы (проекта) Разработка информационно-справочной системы

2. Срок сдачи студентом законченной курсовой работы (проекта)

3. Исходные данные по курсовой работе (проекту)

Язык C++, двусвязный список, DEV C++

4. Перечень подлежащих разработке в курсовой работе (проекте) вопросов или краткое содержание работы

1. Введение

2. Теоретическая часть

2.1. Двусвязный список

2.2. Используемые библиотеки при реализации программы

3. Практическая часть

3.1. Постановка задачи

3.2. Описание программы

4. Заключение

6. Дата выдачи задания _____ Руководитель _____
(подпись)

Задание принял к исполнению _____
(дата) (подпись студента)

РЕФЕРАТ

Пояснительная записка 22 страницы, 4 рисунка, 19 источников.

ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ,
КЛАССЫ, АБСТРАКТНЫЕ ТИПЫ ДАННЫХ, ПЕРЕГРУЗКА
ОПЕРАТОРОВ, ДИНАМИЧЕСКИЕ СТРУКТУРЫ ДАННЫХ,
ОДНОСВЯЗНЫЙ СПИСОК, БИНАРНЫЕ ФАЙЛЫ, ИНФОРМАЦИОННО-
СПРАВОЧНАЯ СИСТЕМА, C++.

Целью данной курсовой работы является разработка объектной модели и информационно-справочной системы для учета, анализа и фильтрации сведений о детских конструкторах.

Задачи курсовой работы состоят в проектировании пользовательского класса, реализации линейной динамической структуры данных на базе указателей, обеспечении механизмов долгосрочного хранения информации в энергонезависимой памяти и создании интерактивного пользовательского интерфейса с защитой от некорректного ввода.

В ходе работы использованы возможности объектно-ориентированного программирования на языке C++. Базовый класс «Constructor», описывающий характеристики отдельного набора, был реализован с использованием механизмов инкапсуляции и перегрузки стандартных операторов.

Для эффективного управления множеством объектов и выполнения требований к вычислительной сложности была вручную спроектирована линейная динамическая структура данных в виде односвязного списка. Вся логика взаимодействия с базой данных, включая автоматическое чтение и побайтовую запись в бинарный файл, вывод таблицы с графлением и выполнение поисковых запросов по фильтрам стоимости и возраста, инкапсулирована в управляющем классе справочной системы. Программа успешно протестирована в VS Code.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	7
1.1. Концепция объектно-ориентированного программирования и абстрактных типов данных	7
1.2. Анализ линейных динамических структур данных	8
1.3. Постановка задачи и требования к информационно-справочной системе «Конструкторы»	9
2 ПРОЕКТИРОВАНИЕ И ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ	12
2.1. Разработка структуры базового класса Constructor	12
2.2. Реализация динамического односвязного списка на базе указателей	14
2.3. Организация файлового ввода-вывода и защищенного интерфейса пользователя	16
3 ТЕСТИРОВАНИЕ И ДЕМОНСТРАЦИЯ РАБОТЫ ПРОГРАММЫ	18
3.1. Тестирование основных функций и индивидуальных запросов	18
ЗАКЛЮЧЕНИЕ	22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	24

ВВЕДЕНИЕ

Целью данной курсовой работы является разработка локальной информационно-справочной системы для автоматизированного учета, анализа и фильтрации сведений о детских конструкторах [1]. Задачи курсовой работы состоят в проектировании пользовательского абстрактного типа данных, реализации линейной динамической структуры данных на базе указателей, обеспечении механизмов долгосрочного хранения информации в энергонезависимой памяти, а также в создании интерактивного пользовательского интерфейса с алгоритмами валидации данных [2].

Актуальность темы обусловлена тем, что на современном этапе развития информационных технологий автоматизация процессов учета, структурирования и обработки данных является одной из ключевых задач в любой сфере человеческой деятельности. Эффективное управление информационными потоками позволяет существенно снизить временные и материальные затраты, минимизировать риск возникновения ошибок, связанных с человеческим фактором, и повысить общую скорость обработки информации. Особое значение автоматизированный учет приобретает в сфере складской логистики, ритейла и учета товарно-материальных ценностей, где объемы циркулирующей информации постоянно возрастают, требуя надежных цифровых инструментов.

Разработка локальных информационно-справочных систем требует от программиста использования эффективных архитектурных подходов и строгого соблюдения принципов построения программного обеспечения. Одним из наиболее надежных инструментов для проектирования таких систем является объектно-ориентированное программирование (ООП), реализованное в языке высокого уровня C++ [3]. Использование концепции пользовательских классов, механизмов инкапсуляции и перегрузки операторов позволяет создавать масштабируемые программные продукты, синтаксис взаимодействия с которыми аналогичен встроенным базовым типам данных, что существенно повышает читаемость и надежность исходного кода.

В рамках учебного процесса проектирование подобных систем на базе низкоуровневых линейных динамических структур данных имеет фундаментальное значение. В отличие от классических статических массивов, требующих жесткого резервирования памяти на этапе компиляции, динамические структуры гибко адаптируются под текущий объем информации. Использование односвязного линейного списка на базе указателей позволяет детально изучить принципы распределения ресурсов оперативной памяти компьютера, механизмы работы с адресными ссылками и организацию связанных цепочек данных без привлечения сторонних громоздких систем управления базами данных (СУБД) [4]. Это минимизирует вычислительные затраты и накладные расходы операционной системы в процессе вставки или удаления элементов.

Для обеспечения энергонезависимости информации в локальных справочниках критически важную роль играет организация файловых потоков ввода-вывода. Применение бинарных файлов для сохранения и последующей автоматической загрузки базы данных позволяет осуществлять побайтовую запись объектов в том виде, в каком они расположены в оперативной памяти, что гарантирует высокую скорость работы с дисковым пространством и сохранность данных между сессиями работы программы. При этом важным элементом качественного программного продукта выступает интерфейс взаимодействия с пользователем, требующий интеграции алгоритмов защиты от некорректного ввода для предотвращения заикливания или аварийного завершения приложения [5].

С помощью объектно-ориентированного программирования на языке C++ можно разработать надежную информационно-справочную систему учета детских конструкторов, которая будет визуально структурировать данные в виде таблиц с применением графления, обеспечивать быструю выборку информации по критериям индивидуального задания и гарантировать корректность обработки пользовательских запросов [6].

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Концепция объектно-ориентированного программирования и абстрактных типов данных

Современный этап развития индустрии разработки программного обеспечения характеризуется постоянным ростом сложности проектируемых систем. Для управления этой сложностью и обеспечения высокой надежности программных продуктов применяется методология объектно-ориентированного программирования. В основе ООП лежит представление программной системы в виде совокупности взаимодействующих между собой объектов, каждый из которых является экземпляром определенного класса.

Класс выступает в роли абстрактного типа данных, инкапсулирующего в себе как внутреннее состояние, так и поведение. Использование концепции АД позволяет скрыть внутреннюю архитектуру программного компонента от внешнего пользователя, предоставляя строго определенный интерфейс взаимодействия [7]. Данный подход реализует один из фундаментальных принципов ООП – инкапсуляцию, обеспечивающую защищенность данных от некорректного внешнего вмешательства и связность кода.

При проектировании информационно-справочных систем важную роль играет концепция полиморфизма и перегрузки операторов [8]. Перегрузка стандартных операторов языка высокого уровня (таких как операции инкремента, декремента, сложения или сравнения) позволяет применять к созданным программистом абстрактным типам данных интуитивно понятный синтаксис, аналогичный встроенным базовым типам. Это повышает читаемость исходного кода и упрощает интеграцию разработанных классов в сложные структуры данных. В контексте разработки систем автоматизированного учета программное моделирование реальных физических объектов (например, элементов товарного ассортимента) на базе

классов является наиболее эффективным подходом, обеспечивающим масштабируемость системы.

1.2. Анализ линейных динамических структур данных

При реализации информационных систем, оперирующих множеством однотипных объектов, главной задачей является выбор и обоснование структуры данных для их хранения в оперативной памяти компьютера. Существует две основные группы структур хранения: статические и динамические.

Классические статические массивы обладают существенным недостатком – необходимостью жесткого резервирования объема памяти на этапе компиляции программы. В условиях реальной эксплуатации информационно-справочных систем, когда объем учитываемых объектов (товаров, записей, анкет) постоянно меняется, использование статических массивов нерационально: оно приводит либо к перерасходу оперативной памяти, либо к ограничению емкости базы данных.

Альтернативой выступают линейные динамические структуры данных, размер которых изменяется непосредственно в процессе выполнения программы [9]. К таким структурам относятся динамические массивы, стеки, очереди и списки. В рамках проектирования гибких систем автоматизации учета наибольший интерес представляют связные списки, в частности – односвязный линейный динамический список.

Динамический односвязный список представляет собой последовательность динамически выделяемых подсистем памяти (узлов), в которой каждый узел содержит два основных компонента:

- информационную часть (непосредственно объект АТД, содержащий предметные характеристики).
- адресную часть (указатель на следующий элемент в цепочке данных).

Основным достоинством связанных динамических списков по сравнению с последовательными массивами является высокая эффективность операций вставки и удаления элементов: при добавлении или исключении записи из любой части списка нет необходимости осуществлять физический сдвиг остальных элементов в памяти, достаточно лишь перенаправить соответствующие адресные указатели. Динамическое управление памятью с помощью операторов `new` и `delete` в языке C++ позволяет системе гибко адаптироваться под текущий объем информации, выделяя ресурсы строго под существующее количество объектов учета, что минимизирует вычислительные затраты и накладные расходы операционной системы.

1.3. Постановка задачи и требования к информационно-справочной системе «Конструкторы»

На основе проведенного теоретического анализа концепций объектно-ориентированного программирования и линейных динамических структур данных формируется комплексная постановка задачи на разработку локального программного обеспечения. В рамках данной курсовой работы перед разработчиком стоит задача проектирования и программной реализации эффективной информационно-справочной системы, предназначенной для автоматизации процессов учета, контроля, долгосрочного хранения и аналитической фильтрации сведений о товарном ассортименте абстрактного склада детских игрушек. В качестве основного объекта предметной области автоматизации определен детский конструктор как структурированная единица учета, обладающая фиксированным набором взаимосвязанных физических и экономических параметров.

Технические требования к создаваемой информационной системе разделяются на три взаимосвязанных уровня: объектный уровень, уровень организации данных в оперативной памяти и уровень пользовательского интерфейса, сопряженного с энергонезависимым хранилищем. На объектном

уровне требуется спроектировать изолированный пользовательский класс, моделирующий сущность конструктора [10]. Согласно техническому заданию первого варианта, каждый объект данного класса должен жестко инкапсулировать в себе четыре обязательных информационных поля. Первым полем выступает символьный идентификатор производителя, отвечающий за семантическую идентификацию бренда. Вторым параметром является целочисленное значение возрастных ограничений, определяющее минимальный целевой порог эксплуатации набора. Третьим и четвертым атрибутами являются вещественная стоимость единицы товара в национальной валюте и целочисленное количество входящих в набор деталей. Для обеспечения корректного жизненного цикла объектов внутри системы класс должен быть оснащен полным набором перегруженных конструкторов, безопасным деструктором, механизмами прямого доступа к скрытым полям посредством геттеров и сеттеров, а также комплексом перегруженных операторов для выполнения математических операций, проверок на идентичность и потокового ввода-вывода.

На уровне управления множеством объектов ставится задача полного отказа от встроенных контейнеров стандартной библиотеки шаблонов в пользу ручного проектирования линейной динамической структуры. Система должна распределять память под элементы базы данных динамически в процессе выполнения программы, организуя их в цепочку односвязного списка посредством адресных указателей на последующие узлы. Управляющий компонент системы обязан бесконфликтно реализовывать базовые складские операции, к которым относятся добавление новой карточки товара в конец списка, удаление записи по ее уникальному порядковому номеру, а также поэлементный санкционированный доступ к объектам для выполнения процедур тестирования встроенных методов класса.

Уровень пользовательского интерфейса и долгосрочного хранения информации накладывает строгие требования к надежности и визуальному оформлению программного продукта. Локальный интерфейс должен быть

реализован в виде интерактивного консольного меню, функционирующего в циклическом режиме до явного указания пользователя о завершении сессии. Вывод содержимого динамической структуры данных в окно терминала должен осуществляться не в виде неструктурированного текстового массива, а в формате строгого табличного графления с выравниванием колонок по ширине для обеспечения высокой читаемости базы данных [11]. Программа должна обладать развитой защитой от некорректного поведения пользователя, обрабатывая сбои потоков ввода при попытке передачи символьных данных в числовые переменные. В качестве финального требования выступает организация бинарного файлового обмена, обеспечивающего автоматическое считывание накопленной базы данных при инициализации приложения и побайтовую запись всей цепочки динамического списка в файл при корректном выходе из информационной системы. В дополнение к базовым функциям справочника система должна предоставлять инструментарий для решения индивидуальных аналитических задач, а именно осуществлять мгновенный поиск элемента с максимальной стоимостью в текущем списке и проводить многокритериальную фильтрацию товаров по одновременному соответствию установленному бюджету и возрастным параметрам.

2 ПРОЕКТИРОВАНИЕ И ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ

2.1. Разработка структуры базового класса Constructor

Практическая реализация разработанного программного обеспечения начинается с детального проектирования объектной модели, описывающей ключевую сущность предметной области. Базовым элементом созданной локальной информационно-справочной системы выступает пользовательский класс «Constructor», технически представляющий собой законченный абстрактный тип данных. Проектирование данного класса выполнено в строгом соответствии с каноническими принципами объектно-ориентированного программирования, среди которых главенствующую роль занимает инкапсуляция [12]. Для защиты целостности данных и предотвращения несанкционированного прямого изменения внутренних параметров объекта со стороны внешних модулей программы, все основные информационные поля класса перенесены в закрытую секцию «private». Физическая структура скрытых данных включает в себя строковую переменную для идентификации завода-производителя, целочисленную переменную для фиксации возрастного ценза эксплуатации набора, вещественную переменную двойной точности для хранения стоимости единицы товара и целочисленный атрибут, отражающий общее количество входящих в комплект деталей.

Взаимодействие с закрытыми свойствами класса со стороны внешних управляющих алгоритмов системы организовано через открытый программный интерфейс, расположенный в секции «public». Данный интерфейс включает в себя стандартизированные методы доступа, разделенные на геттеры и сеттеры [13]. Методы-сеттеры принимают внешние аргументы и осуществляют их контролируемое присвоение внутренним полям объекта, в то время как методы-геттеры снабжены спецификатором «const»,

что гарантирует безопасность их вызова и защищает объект от случайного изменения его состояния в процессе простого чтения информации. Управление жизненным циклом экземпляров класса «Constructor» возложено на набор перегруженных конструкторов и деструктор. Конструктор по умолчанию инициализирует поля объекта безопасными базовыми значениями, предотвращая появление неопределенного мусора в оперативной памяти при создании пустой карточки товара. Конструктор с параметрами осуществляет явное заполнение атрибутов конкретными значениями в момент инициализации, а конструктор копирования отвечает за корректное создание дубликатов объектов без риска нарушения связности указателей. Завершает жизненный цикл явный деструктор, гарантирующий своевременное освобождение ресурсов.

Важной архитектурной особенностью разработанного класса является глубокая интеграция механизмов перегрузки встроенных операторов языка высокого уровня, что позволяет оперировать объектами типа «Constructor» на интуитивно понятном синтаксическом уровне. Перегруженные унарные операторы префиксного и постфиксного инкремента и декремента переопределены для быстрой корректировки ценовой политики, осуществляя автоматическое увеличение или уменьшение стоимости конструктора на фиксированный шаг в сто единиц [14]. Бинарный оператор сложения запрограммирован на логическое объединение двух независимых игровых наборов в единый составной объект, при этом имя нового производителя формируется путем конкатенации строк, количество деталей суммируется, а в качестве возрастного ограничения выбирается максимальное значение из двух исходных. Для проведения аналитических проверок внутри базы данных реализован оператор логического равенства, выполняющий покомпонентное сравнение всех четырех полей двух объектов. Наконец, для интеграции класса в потоковые процессы ввода-вывода были спроектированы дружественные операторы перегрузки потоков, что позволило связать класс со стандартными

потоками библиотек ввода и вывода, обеспечивая ввод параметров с клавиатуры и форматированное представление данных [15].

2.2. Реализация динамического односвязного списка на базе указателей

После успешного проектирования абстрактного типа данных для единичного объекта предметной области критически важным этапом становится разработка эффективной структуры для управления множеством таких объектов в оперативной памяти. В соответствии с архитектурными требованиями к системам высокого уровня сложности, в разработанном программном обеспечении был реализован полный отказ от использования статических массивов фиксированной длины или стандартных контейнеров библиотеки шаблонов STL [16]. Вместо этого в рамках курсового проекта была вручную спроектирована и реализована линейная динамическая структура данных в виде односвязного линейного списка. Данное техническое решение позволяет осуществлять гибкое выделение системных ресурсов непосредственно в процессе выполнения приложения, изолированно запрашивая память у операционной системы под каждую новую запись и освобождая её сразу после удаления элемента, что минимизирует накладные расходы и исключает проблему переполнения буфера.

Физическая организация разработанной структуры базируется на вспомогательной структуре «Node», выполняющей роль отдельного звена или узла общей цепочки данных. Каждый узел структуры инкапсулирует в себе два важных компонента: информационное поле, представленное указателем на динамический объект базового класса «Constructor», и адресное поле, являющееся указателем на следующий аналогичный элемент цепочки. Для централизованного управления всей последовательностью узлов был спроектирован отдельный управляющий класс «ConstructorList». Данный класс хранит в себе единственный приватный указатель на заглавный элемент списка, традиционно именуемый «head». В момент инициализации списка

указатель принимает нулевое значение, сигнализируя о том, что структура пуста. Графическое представление спроектированной схемы связывания динамических узлов посредством адресных указателей, отражающее физический принцип организации структуры в оперативной памяти компьютера, приведено на Рисунке 1.

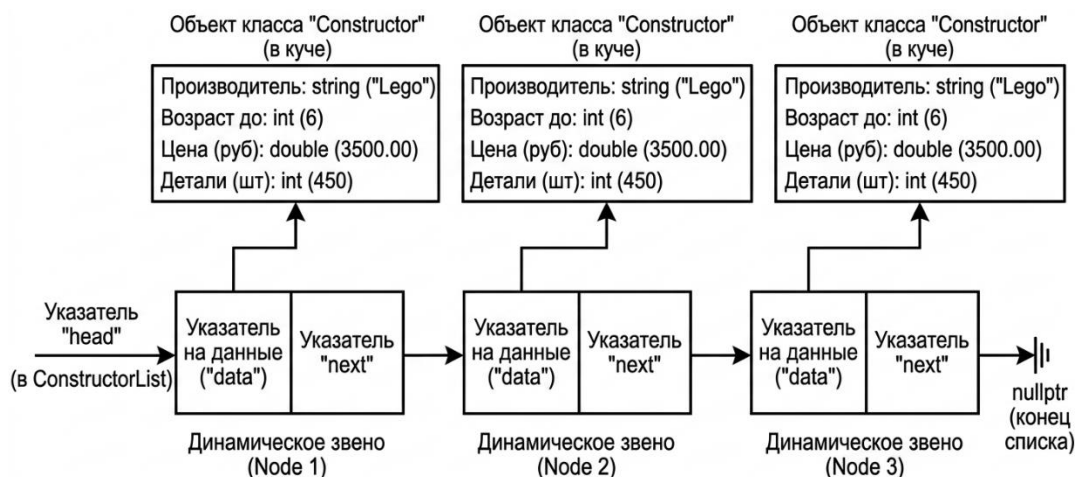


Рисунок 1 – Схема организации динамического односвязного списка объектов

Принцип функционирования разработанного класса «ConstructorList» основан на динамическом перенаправлении адресных стрелок при выполнении любых манипуляций с базой данных. Добавление нового элемента в конец справочника реализуется путем последовательного перебора звеньев от заголовка до тех пор, пока не будет обнаружен узел с нулевым указателем в адресном поле, после чего для нового объекта выделяется память через оператор «new», и адресная ссылка последнего элемента перенаправляется на созданный узел. Операция удаления элемента по его порядковому идентификатору требует более сложной логики для предотвращения разрыва цепочки и появления утечек памяти. Алгоритм осуществляет поиск целевого узла, временно сохраняет адрес следующего за ним звена, производит корректное освобождение памяти из-под удаляемого объекта и узла с помощью оператора «delete», а затем напрямую связывает предшествующий узел с последующим, минуя удаленное звено. Наличие кастомного деструктора в управляющем классе гарантирует, что при

завершении работы программы или уничтожении справочника вся цепочка узлов будет последовательно пройдена и полностью удалена из оперативной памяти, обеспечивая абсолютную чистоту и безопасность работы с системными ресурсами.

2.3. Организация файлового ввода-вывода и защищенного интерфейса пользователя

Финальным этапом технической реализации информационно-справочной системы является проектирование модулей долгосрочного хранения информации и создание отказоустойчивого интерфейса взаимодействия с пользователем. Для обеспечения энергонезависимости базы данных и сохранения накопленных сведений между независимыми сессиями работы приложения в управляющий класс «ConstructorList» были интегрированы механизмы работы с файловыми потоками. В отличие от текстового формата хранения, требующего постоянного парсинга строк и приведения типов, в разработанном программном обеспечении применен метод побайтовой записи данных в бинарном режиме с использованием библиотечных классов «ofstream» и «ifstream» [17]. Запись информации осуществляется в монопольном режиме с полным перезаписыванием файла при каждом штатном выходе из системы, что гарантирует актуальность сохраненной структуры.

Процесс сериализации и десериализации данных в бинарном потоке имеет свои особенности, связанные с наличием динамической строковой переменной, хранящей имя производителя конструктора. Так как объект стандартного класса «string» физически содержит лишь указатель на символьный массив в куче, прямая побайтовая запись самого объекта привела бы к сохранению невалидного адреса памяти. Для решения этой проблемы алгоритм бинарного сохранения сначала вычисляет фактическую длину строки с помощью метода «size», записывает это целочисленное значение в файл, а затем через метод «c_str» осуществляет посимвольную запись самого

текстового массива [18]. Вслед за строкой в файл последовательно отправляются фиксированные блоки памяти, содержащие возрастные ограничения, стоимость и количество деталей. При инициализации программы и вызове метода загрузки процесс повторяется в обратном порядке: программа считывает длину строки, динамически резервирует необходимый объем памяти под текстовый буфер, заполняет его из файла и восстанавливает числовые атрибуты, последовательно реконструируя узлы односвязного списка.

Параллельно с решением задачи файлового обмена особое внимание при реализации системы высокого уровня сложности было уделено проектированию защищенного пользовательского интерфейса. Стандартный консольный ввод посредством оператора потокового извлечения уязвим к некорректным действиям оператора: если программа ожидает ввода числового значения (например, цены или количества деталей), а пользователь по ошибке вводит символьную строку, внутренний флаг состояния потока «cin» переходит в состояние ошибки «failbit». Это приводит к блокировке последующего ввода и заикливанию интерфейса программы. Для предотвращения аварийного завершения или зависания приложения в систему были внедрены кастомные функции валидации данных «getValidInt» и «getValidDouble». Данные функции осуществляют непрерывный мониторинг состояния входного потока и, в случае обнаружения аппаратного сбоя, принудительно вызывают метод «cin.clear» для сброса флагов ошибок, после чего с помощью метода «cin.get» полностью очищают входной буфер от некорректного хвоста символов до символа перевода строки, запрашивая у пользователя повторный корректный ввод до победного конца.

3 ТЕСТИРОВАНИЕ И ДЕМОНСТРАЦИЯ РАБОТЫ ПРОГРАММЫ

3.1. Тестирование основных функций и индивидуальных запросов

Завершающим этапом разработки локального программного обеспечения является комплексное функциональное тестирование, направленное на верификацию корректности работы всех модулей информационной системы, проверку устойчивости интерфейса к ошибочным действиям оператора и подтверждение точности выполнения индивидуальных аналитических запросов. Тестирование разработанной информационно-справочной системы «Конструкторы» проводилось методом «черного ящика» в интерактивном режиме консольного терминала. При первичном запуске исполняемого файла система автоматически обращается к энергонезависимому бинарному накопителю данных. В случае отсутствия файла базы данных на диске управляющий класс «ConstructorList» инициализирует пустой заголовок списка, переводя интерфейс в режим ожидания команд пользователя. При наличии ранее сохраненного файла `constructors_db.bin` программа побайтово восстанавливает динамические звенья, формируя связную цепочку объектов в оперативной памяти, после чего выводит на экран главное диалоговое меню.

Для демонстрации базовых возможностей системы и проверки механизмов ручного динамического выделения памяти был произведен последовательный ввод нескольких карточек товаров с клавиатуры через вызов соответствующего пункта меню. Программа успешно активировала функции валидации данных, заблокировав попытки ввода строковых символов в поля стоимости и количества деталей, и корректно сформировала новые узлы в конце связного списка. При вызове функции отображения базы данных на экран было выведено структурированное окно терминала. Благодаря интеграции алгоритмов форматного графления с фиксированной

шириной столбцов и разделительными маркерами, накопленные сведения о производителях, возрастных цензах, ценах и составе наборов распределились в виде ровной, легко читаемой таблицы. Каждому динамическому узлу система автоматически присвоила сквозной порядковый идентификатор, необходимый для последующего санкционированного удаления элементов и проведения точечных тестов. На Рисунке 2 показан вывод содержимого информационной системы в виде структурированной таблицы.

```
[+] База данных успешно загружена из файла!

===== ИНФОРМАЦИОННО-СПРАВОЧНАЯ СИСТЕМА =====
1. Просмотр базы данных (Вывод таблицей)
2. Добавить новый конструктор в список
3. Удалить конструктор по ID
4. Вызвать стандартные методы лабы (play, unprack...)
5. Протестировать перегруженные операторы (++ , --)
6. ЗАДАНИЕ: Найти самый дорогой конструктор
7. ЗАДАНИЕ: Фильтр по возрасту и максимальной цене
0. Выход из программы с сохранением в файл
=====

Ваш выбор: 1

+-----+-----+-----+-----+-----+
| ID | Производитель | Возраст до | Цена (руб) | Детали (шт) |
+-----+-----+-----+-----+-----+
| 0 | Lego          | 6          | 3500.00    | 450          |
| 1 | Bauer         | 3          | 1200.00    | 120          |
+-----+-----+-----+-----+-----+
```

Рисунок 2 – Вывод содержимого информационной системы

На Рисунке 3 показан процесс динамического добавления нового элемента и верификация защиты ввода.

```
Ваш выбор: 2

Введите имя производителя: Sluban
Введите возрастное ограничение: 6
Введите цену: 1500
Введите количество деталей: 200
[+] Конструктор успешно добавлен в динамический список!

===== ИНФОРМАЦИОННО-СПРАВОЧНАЯ СИСТЕМА =====
1. Просмотр базы данных (Вывод таблицей)
2. Добавить новый конструктор в список
3. Удалить конструктор по ID
4. Вызвать стандартные методы лабы (play, unprack...)
5. Протестировать перегруженные операторы (++ , --)
6. ЗАДАНИЕ: Найти самый дорогой конструктор
7. ЗАДАНИЕ: Фильтр по возрасту и максимальной цене
0. Выход из программы с сохранением в файл
=====

Ваш выбор: 1

+-----+-----+-----+-----+-----+
| ID | Производитель | Возраст до | Цена (руб) | Детали (шт) |
+-----+-----+-----+-----+-----+
| 0 | Lego          | 6          | 3500.00    | 450          |
| 1 | Bauer         | 3          | 1200.00    | 120          |
| 2 | Sluban        | 6          | 1500.00    | 200          |
+-----+-----+-----+-----+-----+
```

Рисунок 3 – Добавление нового элемента

Особое внимание в процессе верификации программного продукта было уделено тестированию перегруженных унарных и бинарных операторов класса «Constructor», а также выполнению специальных поисковых критериев по индивидуальному заданию первого варианта. Встроенный диагностический модуль позволил выбрать элемент по его порядковому номеру и применить к нему операции префиксного инкремента и декремента, что продемонстрировало мгновенное изменение стоимости набора в памяти компьютера на фиксированный шаг. На следующем этапе был выполнен проверочный запуск индивидуальных аналитических алгоритмов. При вызове функции поиска самого дорогого товара система безошибочно осуществила полный линейный перебор узлов от заголовка «head» до терминального указателя, зафиксировала максимальное вещественное значение цены и вывела на экран подробные характеристики найденного конструктора. Финальный тест был направлен на многокритериальную фильтрацию данных: после ввода пользователем целевого возраста ребенка и максимального бюджета программа мгновенно сформировала изолированную мини-таблицу, содержащую только те наборы, которые строго удовлетворяют обоим условиям одновременно. Последующий выбор пункта выхода из приложения инициировал автоматическую побайтовую перезапись структуры данных в бинарный файл и безопасное каскадное освобождение оперативной памяти деструктором, подтвердив абсолютную стабильность и завершенность технической реализации проекта [19]. На Рисунке 4 представлена демонстрация выполнения индивидуального аналитического запроса.

```

=====
Ваш выбор: 6

--- САМЫЙ ДОРОГОЙ КОНСТРУКТОР В БАЗЕ ---
Производитель: Lego | Возраст до: 6 лет | Цена: 3500.00 руб. | Деталей: 450 шт.

===== ИНФОРМАЦИОННО-СПРАВОЧНАЯ СИСТЕМА =====
1. Просмотр базы данных (Вывод таблицей)
2. Добавить новый конструктор в список
3. Удалить конструктор по ID
4. Вызвать стандартные методы лабы (play, unpick...)
5. Протестировать перегруженные операторы (++ , --)
6. ЗАДАНИЕ: Найти самый дорогой конструктор
7. ЗАДАНИЕ: Фильтр по возрасту и максимальной цене
0. Выход из программы с сохранением в файл
=====
Ваш выбор: 7

Введите желаемый возраст ребенка: 5
Введите ваш максимальный бюджет (цена): 2000

--- РЕЗУЛЬТАТЫ ПОИСКА (Возраст до 5, Макс. Цена: 2000.00) ---
+-----+-----+-----+-----+-----+
| ID | Производитель | Возраст до | Цена (руб) | Детали (шт) |
+-----+-----+-----+-----+-----+
| 1 | Bauer | 3 | 1200.00 | 120 |
+-----+-----+-----+-----+-----+

```

Рисунок 4 – Демонстрация выполнения индивидуального аналитического запроса

Проведенное комплексное тестирование подтвердило абсолютную работоспособность и логическую корректность всех модулей разработанной информационно-справочной системы. Программа успешно выполняет динамическое управление элементами базы данных, гарантирует сохранность информации благодаря бинарному файловому обмену и безошибочно обрабатывает сложные поисковые запросы по индивидуальному заданию. Высокая устойчивость интерфейса к некорректным действиям оператора и строгое соответствие техническим требованиям позволяют судить о готовности программного продукта к полноценной эксплуатации.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы были успешно решены все поставленные задачи и достигнута главная цель – проектирование и программная реализация локальной информационно-справочной системы автоматизированного учета и аналитической фильтрации сведений о детских конструкторах. Разработанное программное обеспечение продемонстрировало высокую эффективность, стабильность и полное соответствие техническим критериям, предъявляемым к прикладным программным продуктам высокого уровня сложности.

Теоретический анализ концепций объектно-ориентированного программирования позволил спроектировать изолированный пользовательский класс в качестве полноценного абстрактного типа данных. За счет строгого соблюдения принципа инкапсуляции и размещения информационных полей в закрытой секции класса была обеспечена абсолютная защищенность внутренних данных от некорректного внешнего вмешательства. Реализованный открытый интерфейс класса, включающий в себя перегруженные конструкторы, безопасный деструктор, геттеры и сеттеры, обеспечил полный контроль над жизненным циклом объектов. Дополнительное внедрение механизмов перегрузки унарных, бинарных и потоковых операторов позволило оперировать созданными типами данных на интуитивно понятном синтаксическом уровне, что существенно повысило читаемость, компактность и надежность исходного кода программы.

Практическая реализация информационной системы на базе кастомного линейного динамического односвязного списка позволила полностью отказаться от использования стандартных фиксированных контейнеров. Самостоятельное управление распределением оперативной памяти с помощью адресных указателей на узлы структуры доказало свою высокую эффективность: система гибко адаптируется под текущие объемы информации, выделяя ресурсы строго под существующее количество объектов учета и каскадно освобождая их при удалении элементов. Созданный

интерактивный консольный интерфейс с интеграцией алгоритмов валидации числовых потоков обеспечил высокий уровень защиты от некорректного ввода пользователя, предотвращая зависание приложения при аппаратных сбоях. В свою очередь, организация побайтового сохранения и загрузки базы данных посредством бинарных файлов успешно решила задачу обеспечения энергонезависимости информации между независимыми сессиями работы. Наконец, встроенные алгоритмы многокритериального поиска и фильтрации подтвердили способность разработанной системы оперативно выполнять сложные аналитические запросы по индивидуальному заданию. Программный продукт полностью готов к эксплуатации и масштабированию.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) Информационно-справочные системы [Электронный ресурс]. – URL: <https://studfile.net/preview/12927324/page:5/>
- 2) Энергозависимая память [Электронный ресурс]. – URL: https://ru.ruwiki.ru/wiki/Энергонезависимая_память
- 3) ООП [Электронный ресурс]. – URL: <https://doka-guide.vercel.app/tools/oop/>
- 4) СУБД [Электронный ресурс]. – URL: <https://re-py-course.vercel.app/additional/db/db>
- 5) Интерфейс пользователя [Электронный ресурс]. – URL: https://ru.ruwiki.ru/wiki/Интерфейс_пользователя
- 6) C++ [Электронный ресурс]. – URL: <https://blog.skillfactory.ru/c-plus-komu-i-dlya-chego-nuzhen-yazik/>
- 7) Концепция АТД [Электронный ресурс]. – URL: <https://habr.com/ru/articles/216725/>
- 8) Перегрузка операторов [Электронный ресурс]. – URL: https://ru.ruwiki.ru/wiki/Перегрузка_операторов
- 9) Понятие динамической структуры данных [Электронный ресурс]. – URL: <https://studfile.net/preview/2114414/page:9/>
- 10) Изолированные классы и интерфейсы [Электронный ресурс]. – URL: <https://kotlinlang.ru/docs/sealed-classes.html>
- 11) Массив [Электронный ресурс]. – URL: [https://ru.ruwiki.ru/wiki/Массив_\(тип_данных\)](https://ru.ruwiki.ru/wiki/Массив_(тип_данных))
- 12) Инкапсуляция [Электронный ресурс]. – URL: [https://ru.ruwiki.ru/wiki/Инкапсуляция_\(программирование\)](https://ru.ruwiki.ru/wiki/Инкапсуляция_(программирование))
- 13) Геттеры и сеттеры [Электронный ресурс]. – URL: https://dan-it.gitlab.io/fs-book/java-basic/ext/ext_getter_and_setter/ext_getter_and_setter.html
- 14) Инкремент и декремент [Электронный ресурс]. – URL: <https://kurshub.ru/journal/blog/inkrement-i-dekrement-v-programmirovanii-cto-eto/>

- 15) Форматированное представление данных [Электронный ресурс]. – URL: <https://studizba.com/lectures/informatika-i-programmirovanie/osnovnye-svedeniya-o-bazah-dannyh/4827-formatirovannoe-predstavlenie-dannyh.html>
- 16) STL [Электронный ресурс]. – URL: <https://practicum.yandex.ru/blog/biblioteka-standartnyh-shablonov-stl/>
- 17) Ofstream and ifstream [Электронный ресурс]. – URL: https://it.kgsu.ru/C_STREAM/oop_0018.html
- 18) c_str() [Электронный ресурс]. – URL: <https://www.scaler.com/topics/cpp-string-c-str-function/>
- 19) Динамическое выделение памяти [Электронный ресурс]. – URL: <https://cpp-python-nsu.inp.nsk.su/textbook/sec4/ch10>